Performance Measurement of a Web Application

CS 681 Assignment 1
Team Arjun

Shikhar Rai (25M2128)
Viren Mehta (25M2132)

System Specifications

Server Machine (Windows)

- OS: Microsoft Windows 11 Home (64-bit)
- CPU: 13th Gen Intel(R) Core(TM) i5-1340P, 12 cores / 16 threads
- RAM: 16 GB
- Web Server: Apache
- Scripting: PHP

Client (Load Generator) Machine (Linux)

- OS: Ubuntu 24.04.3 LTS
- CPU: Intel(R) Core(TM) Ultra 7 255H, 16 cores / 16 threads
- RAM: 14 GB
- Load Tool: Tsung, httperf

Open Loop

Open load testing is performed in a client–server setup, where a client tool sends requests to an Apache web server running PHP at a fixed arrival rate.
This experiment helps observe how the server handles increasing load, and how its response time and throughput change as the server approaches and exceeds its capacity.

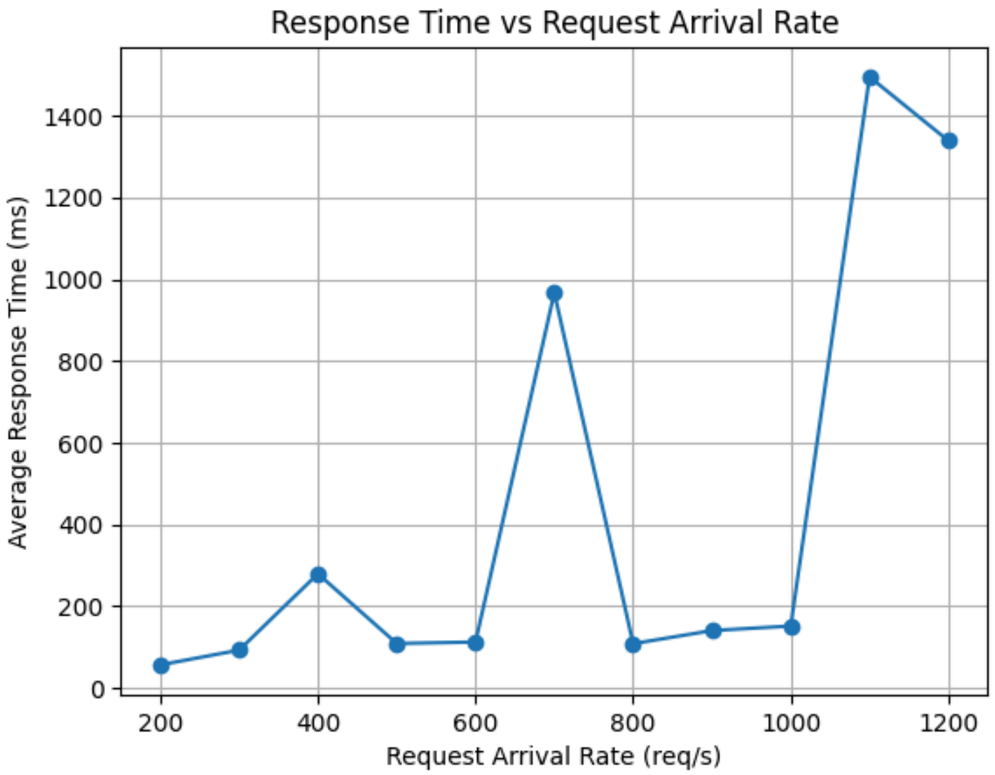
Arrival Rate defines offered load levels

- Each value = requests per second
- This is the open-loop arrival rate λ

Number of Connections	5000
Arrival Rates	200 300 400 500 600 700 800 900 1000 1100 1200

- A script was used to run multiple httperf tests at different request arrival rates.
- Each run generated load on the server and stored the corresponding results for analysis.

Response Time vs Request Arrival Rate

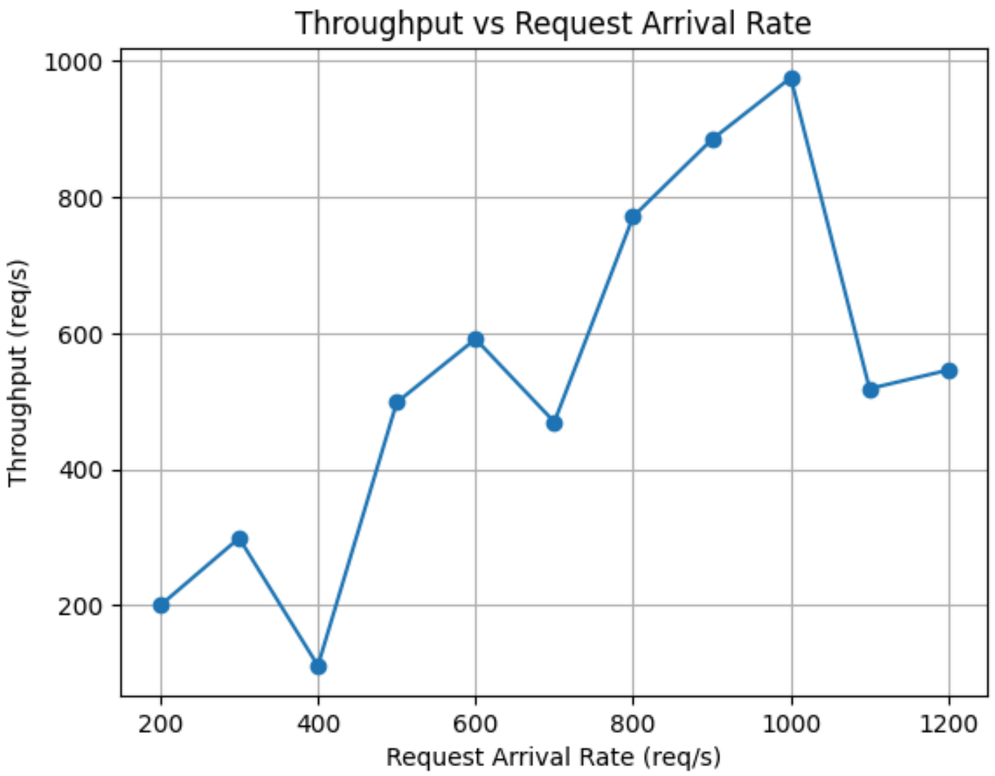


The response time increases slowly at low arrival rates and then increases sharply as the request arrival rate approaches the system's maximum capacity.

- Minimum response time is observed at low load≈ 200–300 req/s.
- Response time becomes very large near saturation (≈ 1000 req/s and beyond).

- At low load, the server processes requests immediately, so response time is dominated by service time.
- As the arrival rate increases and approaches the service capacity of the server, packets begin to drop and delay increases rapidly, leading to a sharp rise in response time.

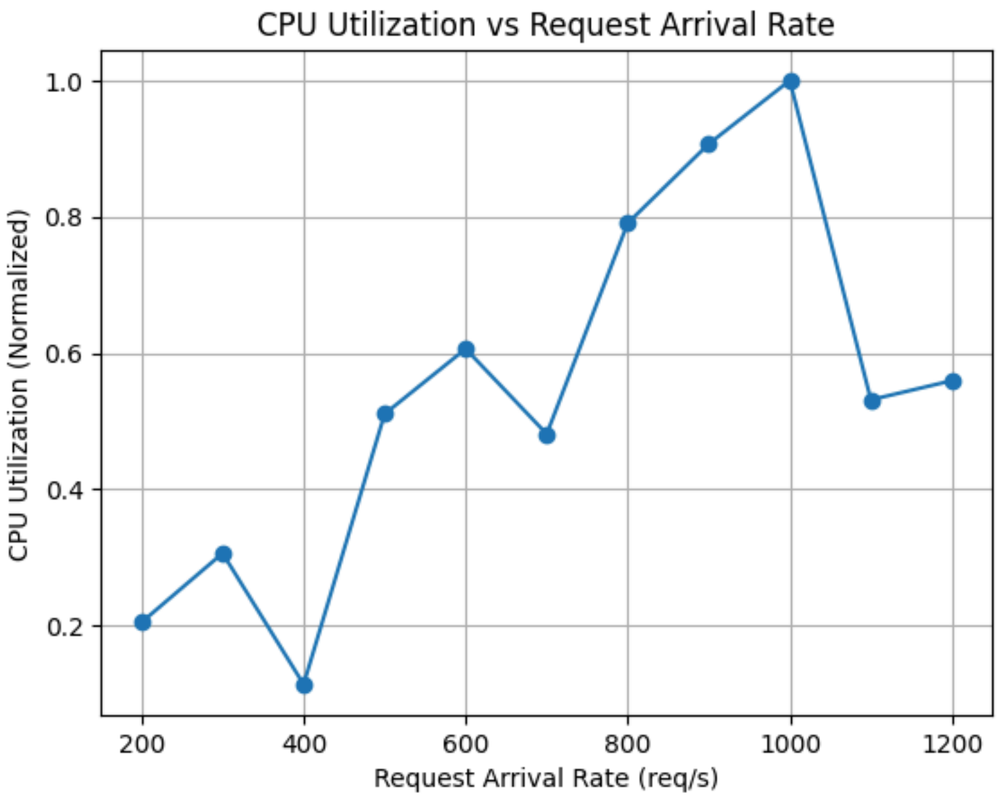
Throughput vs Request Arrival Rate



- Throughput increases linearly at low arrival rates.
- Throughput saturates at approximately **1000 req/s.**
- At very high arrival rates, throughput decreases.

- When the system is lightly loaded, it is able to service all incoming requests, so throughput equals the arrival rate.
- Once the server reaches its maximum processing capacity, it cannot handle additional load.

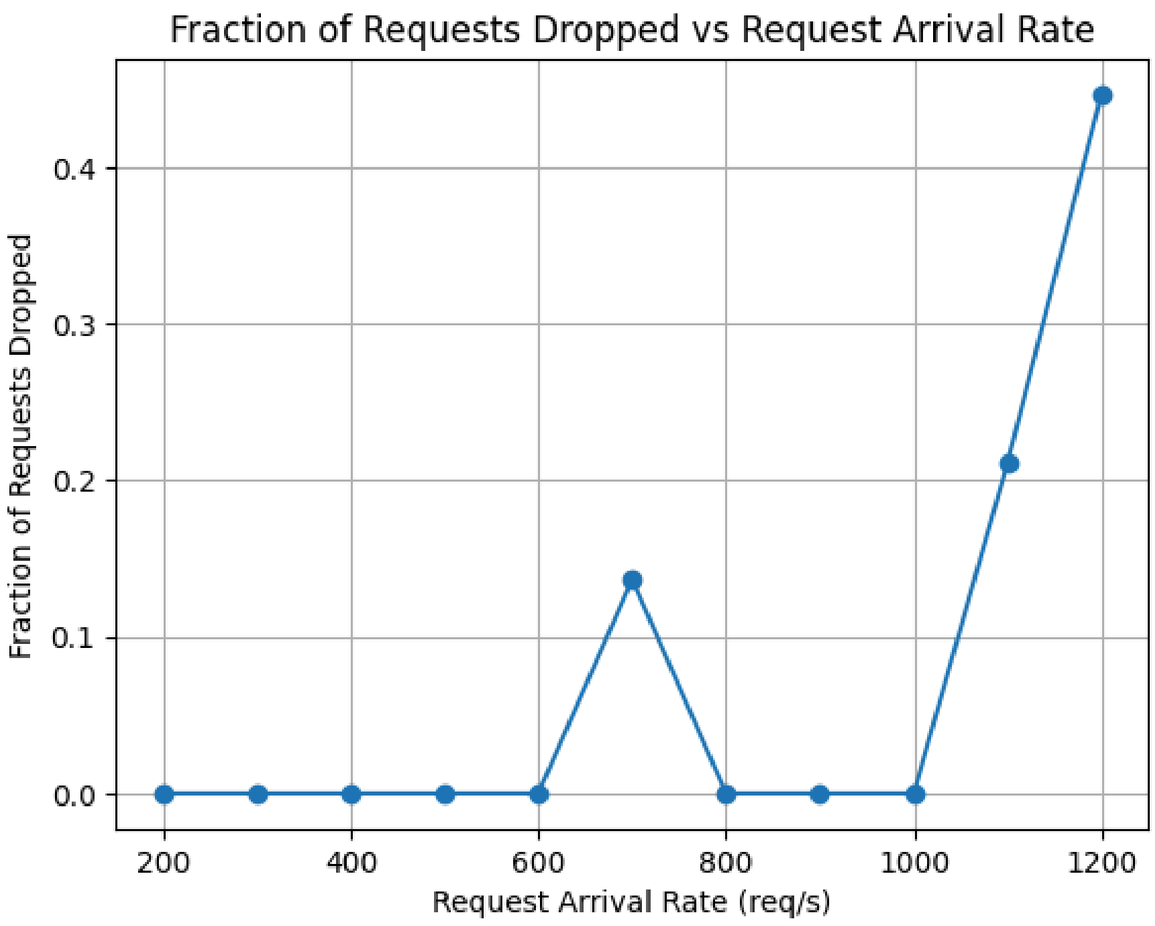
Server CPU Utilization vs Request Arrival Rate



- Utilization grows roughly linearly before saturation.
- Utilization remains close to maximum once the server is saturated
- Single dominant CPU bottleneck
- Near saturation, U ≈ 1

- According to the Utilization Law, CPU utilization is proportional to throughput. As the arrival rate increases, throughput increases, leading to higher CPU utilization.
- Once the CPU becomes the bottleneck, utilization approaches 100% and cannot increase further, even if the arrival rate continues to grow.

Fraction of Requests Dropped vs Request Arrival Rate



Arrival Rate	Requests	Replies	Fraction Dropped
200-600	5000	5000	0.0
700	5000	4317	0.13
800-1000	5000	5000	0.0
1100	5000	3944	0.21
1200	5000	2767	0.44

Fraction of Requests Dropped vs Request Arrival Rate (Contd)

The fraction of dropped requests remains zero at low arrival rates and then increases sharply once the system becomes overloaded.

- **Zero request drops below saturation.**
- **Rapid increase in dropped requests beyond saturation.**

At low load, sufficient server resources (workers, connection backlog) are available to accept all incoming requests. Once these resources are exhausted, the server begins refusing new connections, leading to an increase in dropped requests. This behavior indicates **overload** rather than application failure.

Maximum Load Supported by the Server

The highest request arrival rate at which:

- Throughput closely matches arrival rate,
- Response time remains bounded,
- Fraction of dropped requests is approximately zero

Maximum sustainable request arrival rate supported by the server is approximately 900–1000 requests per second.
Beyond this point, response time increases sharply and requests begin to be dropped.

Service Time Estimation Using the Utilization Law

$$U = X \times S$$

Where:

- U is CPU utilization,
- X is throughput (requests/sec),
- S is average service time per request.

Estimation At saturation:

- U≈1, (max out after 1000<<<<reqs/sec)
- Peak stable throughput (X_max): ≈ 900–1000 reqs/sec
  Using the Utilization Law the service time is 1/X=S, 1/1000≈1msec
- Near saturation, the system holds about 1000 requests, giving a peak response time
  W=L/X=1000/1000=1 second.
- Average response time at overload or after saturation: ≈1000–1400 ms ≈ 1 second
- Total requests per run: 5000
- Replies received: varies by load

Open-Load Experiment Summary Table

Arrival Rate (req/s)	Requests	Replies	Throughput (req/s)	Avg Response Time (ms)	Max Concurrent Conn.	Total Errors	Fraction Dropped
200	5000	5000	199.6	25.2	54	0	0
300	5000	5000	298.5	52.8	106	0	0
400	5000	5000	110.8	60.1	185	0	0
500	5000	5000	498	50.9	147	0	0
600	5000	5000	591.2	58.7	183	0	0
700	5000	4317	469.2	616.9	1316	683	**0.1366**
800	5000	5000	771.2	58.5	211	0	0
900	5000	5000	884.6	79.2	253	0	0
1000	5000	5000	975.4	76.2	316	0	0
1100	5000	3944	518.2	1000.7	2672	1056	**0.2112**
1200	5000	2767	545.6	943.1	2441	2233	**0.4466**

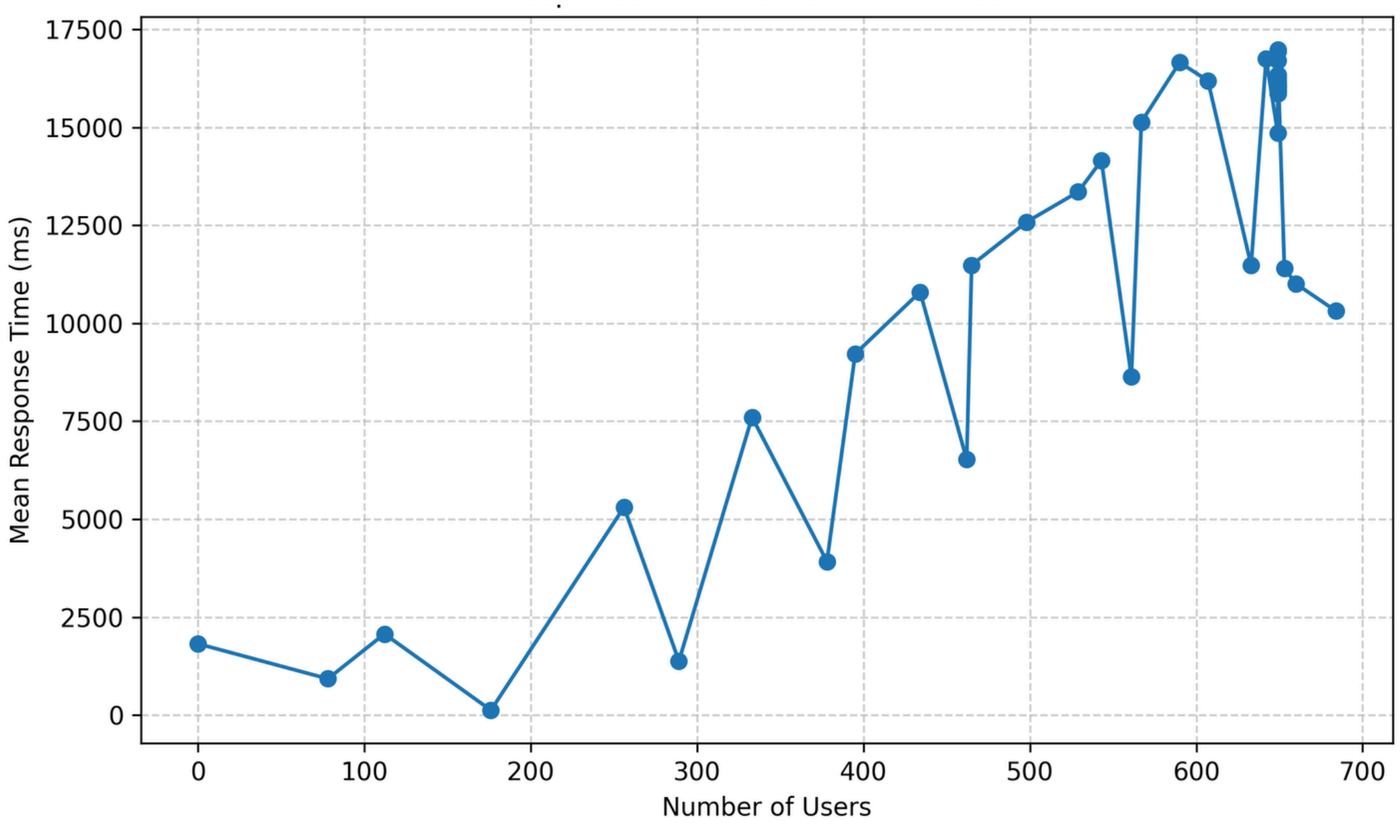
Closed Load

This part focuses on evaluating the performance of a web server under load using closed-loop workload modeling. A PHP-based application hosted on an Apache server was stress-tested using **Tsung** to analyze **response time, throughput, utilization, and saturation behavior.**

Parameters	
duration	1200 s
maxnumber	1000
arrivalrate	10 users/sec
thinktime	6 s

A closed-loop workload defined in an XML configuration. The setup specifies a maximum of **1000 concurrent users**, a ramp-up rate of 10 users per second, a finite user lifetime of **25 request iterations**, and a think time of 6 seconds.

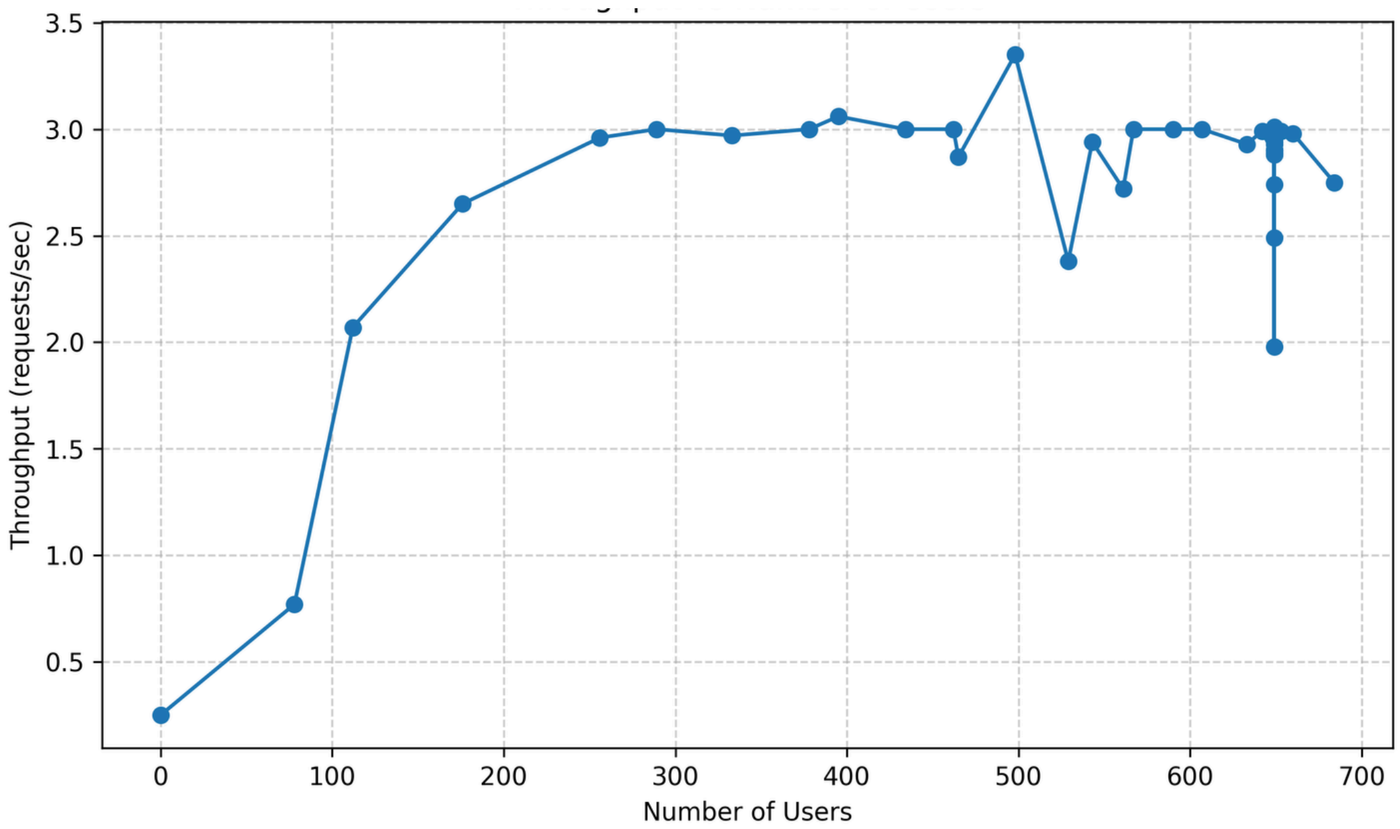
Response time vs Number of users



Minimum RT ≈ 200–300 ms
Maximum RT ≈ 16–17 seconds

- When the number of users is small, response time stays low because requests are served quickly with little waiting.
- Adding user adds more waiting time, with the lowest response time at low load and the highest at heavy load.

Throughput vs Number of users

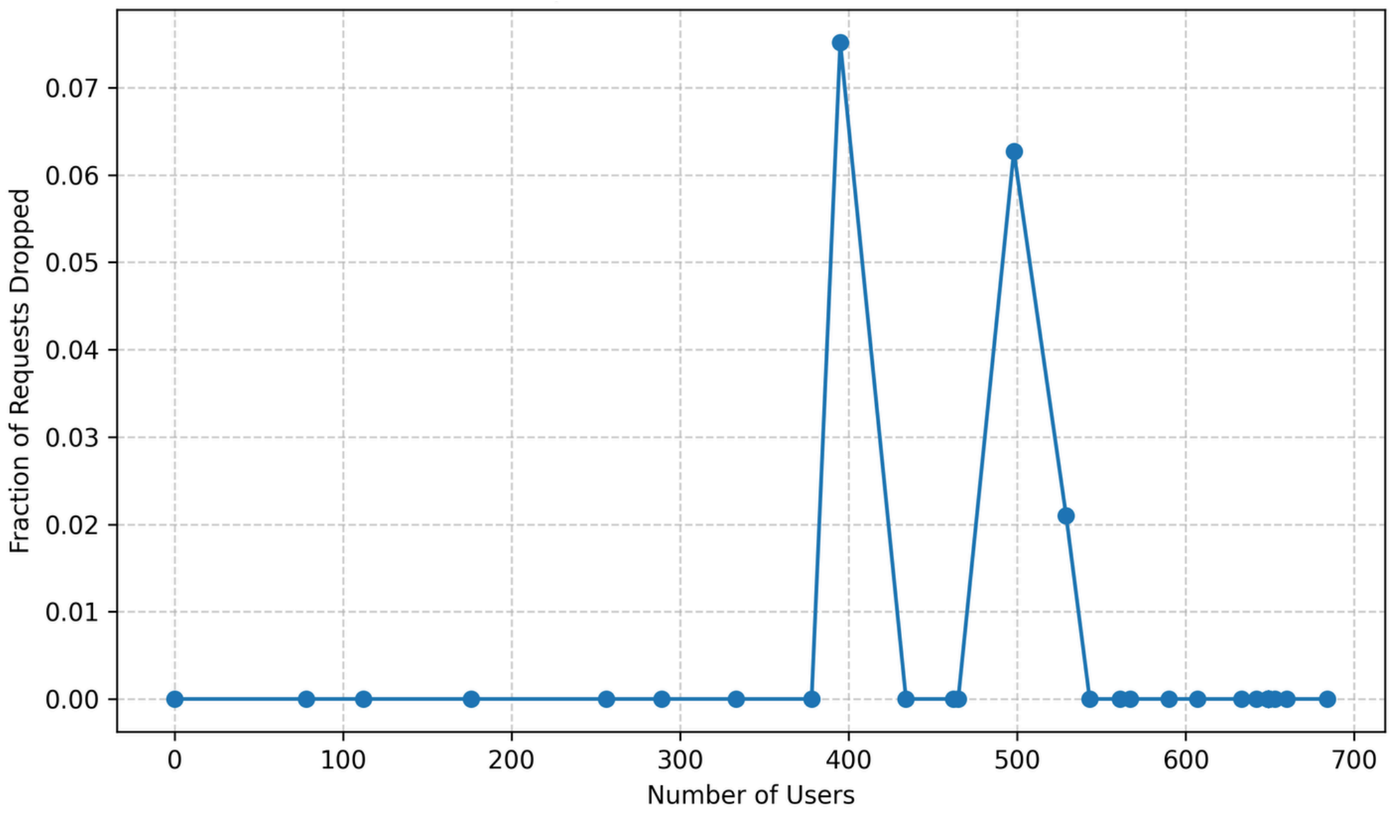


Maximum throughput ≈ 3 req/sec

Once the bottleneck is saturated (around 300 users), throughput does not increase further

- As the number of users increases, throughput rises at first because more requests are being processed in parallel, but after a certain point it stops increasing and levels off as the server reaches its capacity.
- The maximum throughput is about **3 requests per second**; beyond this point, adding more users only leads to overload-related fluctuations.

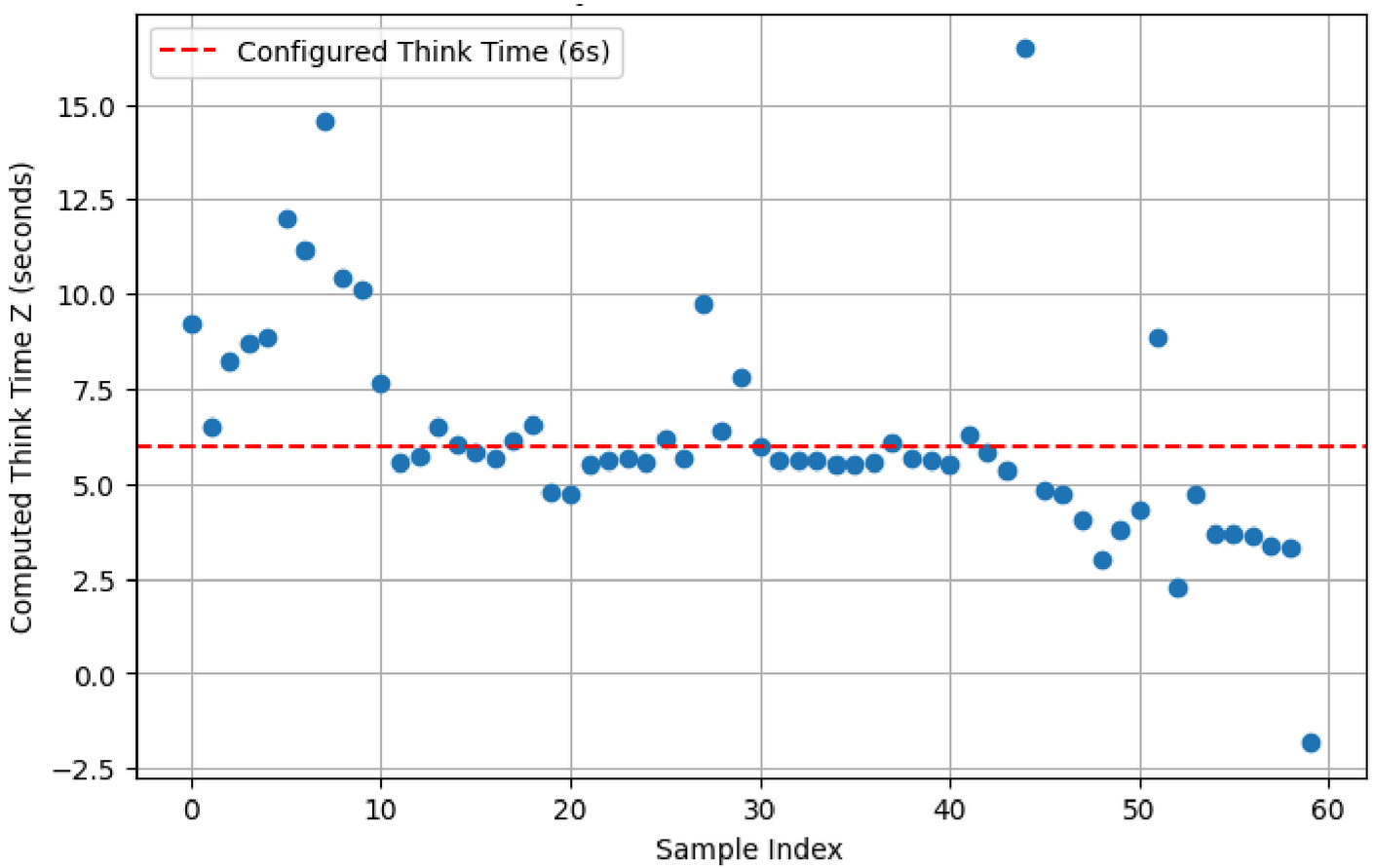
Drop Fraction vs Number of users



Small spikes near saturation
Server handles normal load

- At low and moderate load, almost no requests are dropped because the server can handle the traffic without any problems.
- Near overload, a few requests get dropped due to timeouts or queue limits, but the server remains stable

Verification of Closed-System Little's Law



For a large portion of the run, computed thinktime(Z):

- **Clusters tightly around 6 s**
- Matches the configured think time

This confirms that closed-system Little's Law is verified during steady state.

$$Z=(N/X)-R$$

- N is number of users
- X is throughput
- R is average response time

What the plot shows:

- Each dot represents one measurement interval
- Y-axis: Think time computed using closed-system Little's Law
- Red dashed line: Configured think time (Z_0 = 6 s)
- X-axis: Measurement interval index (time-ordered samples)

Little’s Law for Closed System

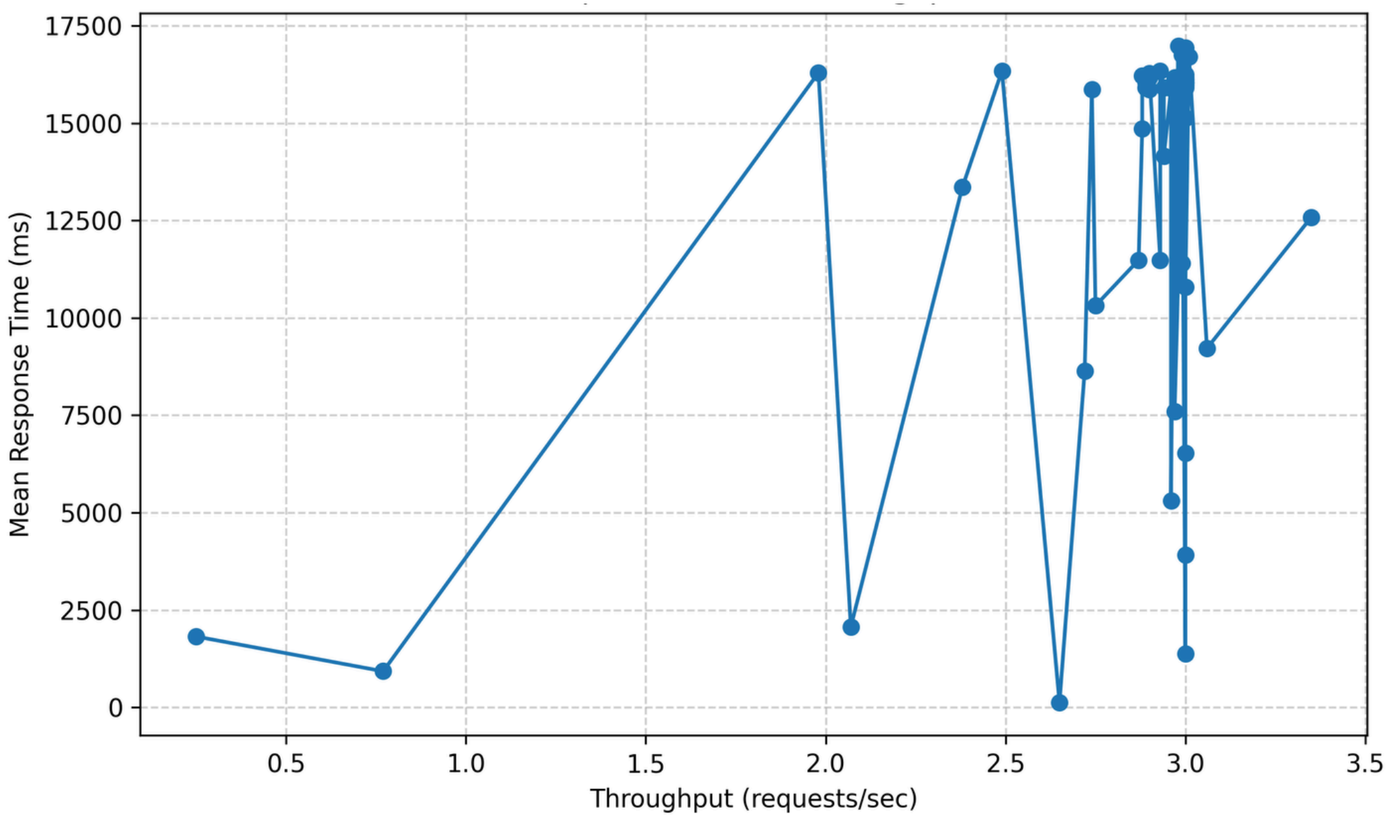
$$N=X(R+Z)$$

N = number of concurrent users
X = throughput (requests/sec)
R = response time
Z = think time (constant)

Throughput	Response Time
0.77 req/sec	926.9 ms
2.65 req/sec	124.1 ms
3.00 req/sec	1379.7 ms
3.00 req/sec	3920.0 ms
3.00 req/sec	6522.9 ms

- Before saturation, increasing users increases throughput while response time remains low.
- After saturation, throughput remains nearly constant while response time increases sharply, matching the inverse relationship predicted by Little’s Law.

Response time vs Throughput



Throughput stabilizes near 3 req/sec
Response time grows up to ~17 sec

- As throughput increases, response time stays low at first, but once throughput approaches its maximum, response time rises sharply because the server is becoming saturated.
- Beyond this point, throughput does not increase much, but response time varies widely and becomes very high, showing queue buildup and unstable performance near saturation.